VS Code에서 Codex와 Claude Code 사용하기

AI 코딩 확장 프로그램 설치부터 첫 프로젝트 실습까지

(1) 교재구성

1장	개발 환경 준비	1.1 VS Code 설치/ 1.2 계정과Node.js 준비/ 1.3 환경 점검 실습
2장	VS Code에서Codex 사용하기	2.1 설치와 로그인/ 2.2 권장 설정/ 2.3 첫 프롬프트/ 2.4 명령 승인과 결과 확인
3장	VS Code에서Claude Code 사용하기	3.1 사전 조건과 설치/ 3.2 시작하기/ 3.3 CLAUDE.md로 프로젝트 안내하기
4장	실습: 워드패드 클론과 두 도구 비교	4.1 프로젝트 구조/ 4.2 소스 코드/ 4.3 실행과 확인/ 4.4 비교와 문제 해결

(2) 소스코드와 표기규칙

모든 소스 코드는 함께 제공되는 `vibe_coding.zip`에 교재와 같은 경로로 들어 있습니다 (`ch01/check_env.js`, `ch02/prompts`, `ch03/prompts`, `ch04/wordpad-clone`). `D:\vibe_coding`에 풀면 교재의 경로와 프롬프트가 그대로 일치하며, macOS와 Linux에서는 `~/vibe_coding`에 풀고 프롬프트의 `D:\vibe_coding\ch01>`를 `~/vibe_coding/ch01$`로 바꿔 읽습니다. 파일 경로, 코드, 터미널 명령은 `D2Coding` 회색 상자로 표시하며, 터미널 상자에서 `>`뒤는 입력할 명령, 그 아래 줄은 실행 결과입니다. 버전이나 시간 같은 값은 환경에 따라 다릅니다.

1장 개발 환경 준비

이 장에서는 바이브 코딩, 즉AI에게 자연어로 지시하며 프로그램을 만드는 방식을 시작하기 전에 필요한 개발 환경을 준비합니다. 코드 에디터인VS Code를 설치하고, AI 도구를 사용할 계정과 웹 앱 실행에 필요한Node.js를 갖춘 뒤, 간단한 스크립트로 환경이 정상인지 점검합니다.

학습목표

- VS Code의 역할을 이해하고 운영체제에 맞게 설치할 수 있다.
- Codex와Claude Code에 필요한 계정 조건을 구분할 수 있다.
- Node.js를 설치하고 터미널에서 버전을 확인할 수 있다.
- check_env.js를 실행하여 실습 환경을 스스로 점검할 수 있다.

(3) 이장의구성

1.1	VS Code 설치	VS Code 소개, 운영체제별 설치 순서
1.2	계정과Node.js 준비	ChatGPT·Anthropic 계정 조건, Node.js 설치와 확인
1.3	환경 점검 실습	작업 폴더 생성, check_env.js 작성과 실행

1.1 VS Code 설치

이 절에서는VS Code가 무엇인지 알아보고, 운영체제에 맞는 설치 파일을 내려받아 설치합니다.

(1) VS Code란

VS Code(Visual Studio Code)는Microsoft가 만든 무료 코드 에디터입니다. 가벼운 에디터이지만 확장 프로그램을 설치하면 코드 작성, 실행, 디버깅까지 할 수 있어IDE(통합 개발 환경)처럼 쓰입니다. 이 교재에서는Codex와Claude Code를 모두VS Code의 확장 프로그램으로 설치하여 사용합니다.

운영체제	지원 여부	설치 파일 형식
Windows	지원(10 이상 권장)	User Installer(.exe) 64-bit
macOS	지원	.zip 압축 해제 후Applications 폴더로 이동
Linux	지원	.deb, .rpm 패키지 또는snap

(2) 설치 순서

설치 과정은 일반 프로그램과 다르지 않습니다. 아래 순서를 따라 합니다.

① 설치 파일 내려받기

브라우저에서 공식 사이트 `code.visualstudio.com`에 접속하여 Download 버튼으로 운영체제에 맞는 버전을 내려받습니다. 공식 사이트가 아닌 곳의 설치 파일은 받지 않습니다.

② 설치 마법사 실행

Windows에서는 내려받은.exe를 실행하고'동의합니다'를 선택한 뒤 다음을 누릅니다. 추가 작업 선택 화면에서는'PATH에 추가'(기본 선택)를 그대로 두고, 가능하면'Code(으)로 열기 작업을 탐색기 상황에 맞는 메뉴에 추가'도 선택합니다.

③ 실행 확인

설치가 끝나면 VS Code를 실행합니다. 환영 화면이 나타나면 설치가 완료된 것입니다. 한국어 화면이 필요하면 확장 보기(`Ctrl+Shift+X`, macOS는 `Cmd+Shift+X`)에서'Korean Language Pack for Visual Studio Code'를 설치한 뒤 재시작합니다.

1.2 계정과 Node.js 준비

이 절에서는 AI 도구를 쓰기 위한 계정 조건을 정리하고, 웹 앱을 실행하는 데 필요한Node.js를 설치합니다.

(1) 필요한 계정

두 도구는 서로 다른 회사의 계정으로 로그인합니다. 하나만 사용해도 되지만, 이 교재의 비교 실습을 위해서는 둘 다 준비하는 것이 좋습니다.

도구	로그인 계정	비고
Codex	ChatGPT(OpenAI) 계정	ChatGPT를 쓰고 있다면 새로 가입할 필요가 없습니다. 이용 가능한 요금제와 사용 한도는 자주 바뀌므로 공식 안내를 확인합니다.
Claude Code	Anthropic 계정	유료Claude 구독(Pro, Max, Team, Enterprise) 또는 Claude Console 계정이 필요하며API 키는 필요 없습니다.

(2) Node.js 설치

React와 Vite 프로젝트는 `npm install`, `npm run dev` 같은 명령으로 패키지를 설치하고 실행합니다. 이 명령들은 Node.js(npm 포함)가 있어야 동작하므로 먼저 설치합니다. AI 도구가 자동으로 설치해 주지 않으므로 직접 준비해야 합니다.

`nodejs.org`에서 LTS 버전(20 이상)을 내려받아 기본 옵션으로 설치합니다. 이 교재의 프로젝트는 Node.js 18 이상이면 동작합니다. 설치 후 새 터미널을 열어 아래 두 명령으로 버전을 확인합니다.

터미널(명령 프롬프트)

```
D:\vibe_coding> node -v
v20.18.0
D:\vibe_coding> npm -v
10.8.2
```

`node -v`는Node.js의 버전을, `npm -v`는 패키지 관리자 npm의 버전을 출력합니다. '인식할 수 없음' 오류가 나면 VS Code와 터미널을 닫았다가 다시 엽니다.

1.3 환경 점검 실습

이 절에서는 작업 폴더를 만들고, Node.js로 간단한 점검 스크립트를 실행하여 환경이 올바른지 확인합니다.

(1) 작업 폴더 준비

`D:\vibe_coding\ch01`폴더를 만들고VS Code로 엽니다(`code .`또는 `File > Open Folder`). 이후 파일은 모두 이 폴더 아래에 둡니다.

(2) 점검 스크립트 작성과 실행

`ch01`폴더에 아래 파일을 만들고 `node check_env.js`로 실행합니다.

ch01/check_env.js

```
// 실습 환경 점검 스크립트: Node.js 버전과 운영체제를 확인한다.
const [major] = process.versions.node.split(".").map(Number);

console.log("Node.js 버전:", process.version);
console.log("운영체제:", process.platform);

if (major < 18) {
  console.log("결과: Node.js 18 이상으로 업데이트해야 합니다.");
  process.exitCode = 1;
} else {
  console.log("결과: 실습 환경을 사용할 수 있습니다.");
}
```

첫 줄은 주석입니다. `process.versions.node`를 `.`으로 나누어 숫자로 바꾸고 첫 값(주 버전)을 `major`에 담습니다. `console.log`는 기본 출력문이고, 주 버전이 18 미만이면 안내 후 종료 코드를 1로 설정합니다.

터미널(명령 프롬프트)

```
D:\vibe_coding\ch01> node check_env.js
Node.js 버전: v20.18.0
운영체제: win32
결과: 실습 환경을 사용할 수 있습니다.
```

2장 VS Code에서Codex 사용하기

이 장에서는 OpenAI의 코딩 에이전트 Codex를 VS Code 확장 프로그램으로 설치하고, 로그인과 기본 설정을 마친 뒤 첫 프롬프트로 프로젝트를 만들어 봅니다. Codex는 내 컴퓨터에서 실행되므로 터미널 명령을 실행하기 전에 승인을 요청하는데, 이 승인 흐름도 함께 익힙니다.

학습목표

- Codex 확장 프로그램을 설치하고ChatGPT 계정으로 로그인할 수 있다.
- 모델, 추론 노력, 실행 권한 설정의 의미를 설명할 수 있다.
- 구체적인 프롬프트를 작성하여 프로젝트 생성을 지시할 수 있다.
- 터미널 명령 승인 요청을 확인하고 결과를 브라우저에서 확인할 수 있다.

이장의구성

2.1	설치와 로그인	확장 설치, Windows 참고 사항, 계정 로그인
2.2	권장 설정	모델, 추론 노력, 실행 권한
2.3	첫 프롬프트	프롬프트 작성 원칙, 예시 프롬프트
2.4	명령 승인과 결과 확인	승인 흐름, 개발 서버 실행과 확인

2.1 설치와 로그인

이 절에서는 Codex 확장을 설치하고 ChatGPT 계정으로 연결합니다. 화면 구성은 업데이트에 따라 조금씩 달라질 수 있습니다.

(1) 확장 프로그램 설치

① 웹에서 설치하기

- 브라우저에서 `openai.com/codex`에 접속합니다.
- ‘IDE에서 사용하기(Try in your IDE)’ 버튼을 클릭합니다.
- IDE 선택 화면에서 VS Code를 선택합니다.
- VS Code 확장 마켓플레이스 페이지가 열리면 Install(설치)을 누릅니다.
- 설치가 끝나면 VS Code 왼쪽 사이드바에 Codex 아이콘이 생깁니다.

2 VS Code에서 직접 설치하기

VS Code에서 `Ctrl+Shift+X`(macOS는 `Cmd+Shift+X`)로 확장 보기를 열고‘Codex’를 검색합니다. 게시자가 OpenAI 인지 확인한 뒤 설치합니다. 이름이 비슷한 다른 확장이 많으므로 게시자 확인이 중요합니다.

(2) Windows 사용자 참고

Codex VS Code 확장은 macOS와 Linux에서 안정적으로 지원되며, Windows 지원은 실험적(experimental) 단계로 안내되어 왔습니다. Windows에서는 WSL(Windows Subsystem for Linux)에서 사용하는 것이 권장됩니다. 지원 상태는 바뀔 수 있으므로 공식 문서를 함께 확인합니다. 아래는 PowerShell을 관리자 권한으로 열어 WSL을 설치하는 명령입니다.

터미널(PowerShell, 관리자 권한)

```
PS C:\> wsl --install
PS C:\> wsl --list --online
```

`wsl –install`은 기본 Linux 배포판(Ubuntu)과 WSL을 설치하며, 설치 후 컴퓨터를 다시 시작해야 합니다. `wsl --list --online`은 설치 가능한 배포판 목록을 보여 줍니다. 설치 후 VS Code에 ‘WSL’ 확장을 설치하고 왼쪽 아래 ><버튼에서‘Connect to WSL’을 선택하면 Linux 환경에서 폴더를 열 수 있습니다. 이때 Node.js도 WSL 안에 다시 설치해야 합니다.

(3) ChatGPT 계정으로 로그인

사이드바의 Codex 아이콘을 클릭하면 로그인 안내가 나타납니다. ChatGPT 계정(이메일과 비밀번호 또는 연결된 소셜 계정)으로 로그인하고 브라우저의 인증을 완료하면 Codex 패널이 활성화되고 프롬프트 입력창이 나타납니다. 이용 가능 여부는 계정의 요금제에 따라 다릅니다.

2.2 권장 설정

이 절에서는 Codex 패널의 설정 항목을 살펴보고, 처음 사용할 때 권장하는 값을 정합니다. 처음에는 기본값으로 충분합니다.

(1) 모델

이 교재가 참고한 자료에서는 GPT-5.3-Codex를 권장합니다. 같은 확장에서 GPT-5.3-Codex-Spark도 선택할 수 있는데, Spark는 거의 실시간에 가까운 응답 속도를 목표로 하는 소형 모델입니다. 처음에는 GPT-5.3-Codex를 쓰고, 빠른 응답이 필요할 때 Spark를 시도해 봅니다. 모델 목록은 자주 바뀌므로, 선택 목록에서 Codex 이름이 붙은 최신 모델을 고르면 됩니다.

(2) 추론 노력(Reasoning Effort)

추론 노력은 모델이 답을 내기 전에 얼마나 깊이 생각할지를 정합니다. Low는 빠르지만 결과가 단순해질 수 있고, High는 복잡한 문제에 유리하지만 시간이 오래 걸립니다. 처음에는 Medium으로 설정합니다.

(3) 실행 권한

실행 권한은 Codex가 어디까지 스스로 할 수 있는지를 정합니다. 기본값은 로컬 프로젝트 폴더 안에서만 작업하도록 제한되므로 시스템 전체에 영향을 주지 않아 안전합니다. 폴더 밖의 파일을 바꾸거나 네트워크를 쓰는 작업은 사용자에게 먼저 묻습니다. 모든 권한을 허용하는'Full Access'류의 모드는 익숙해지기 전까지 사용하지 않습니다.

2.3 첫 프롬프트

이 절에서는 Codex에게 전달할 첫 번째 프롬프트를 작성합니다. 프롬프트란 Codex에게 전달하는 작업 지시문으로, 웹의 ChatGPT에 입력하던 것과 같은 방식입니다.

(1) 프롬프트 작성 원칙

- **기능을 구체적으로 쓴다.**'웹앱 만들어줘'보다'React로 파일 저장/열기 기능이 있는 워드패드 클론'처럼 명시합니다.
- **기술 스택을 적는다.**React, Vue, 순수 HTML 등을 지정하지 않으면 Codex가 임의로 선택합니다.
- **영어를 권장한다.** 영어가 더 안정적인 경우가 많지만 한국어도 쓸 수 있으며, 한국어로 쓸 때는 파일명과 명령어를 영어 그대로 적습니다.

(2) 예시 프롬프트

아래 프롬프트는 이 교재의 실습 프로젝트를 만들기 위한 것입니다. 원문의 예시에 밑줄 기능과 제약 조건, 폴더 이름을 더하여 결과가 예측 가능하도록 했습니다.

ch02/prompts/first_prompt.txt

```
Create a Wordpad clone in React (Vite) with bold, italic, underline,
font size controls, and file save/open functionality.
Use only React and Vite. Do not add any other UI library.
Put the project in a folder named wordpad-clone.
```

첫 줄은 만들 대상(React와 Vite 기반 워드패드 클론)과 기능(굵게, 기울임, 밑줄, 글자 크기, 저장/열기)을 지정합니다. 셋째 줄은 외부UI 라이브러리를 쓰지 않도록 제한하여 결과를 단순하게 만들고, 넷째 줄은 프로젝트 폴더 이름을 지정합니다. 이 정도 지시만으로도 Codex는 프로젝트 폴더를 만들고 필요한 패키지를 설치하며 코드를 자동으로 생성합니다.

2.4 명령 승인과 결과 확인

이 절에서는 Codex가 터미널 명령을 실행하려 할 때 나타나는 승인 요청을 이해하고, 완성된 앱을 브라우저에서 확인하는 방법을 익힙니다.

(1) 명령 승인 흐름

Codex가 코드 생성 외에 `npm install`, `npm run build` 같은 터미널 명령을 실행해야 할 때는 먼저 명령을 보여 주고 승인을 요청합니다. 예를 들어'터미널에서 다음 명령을 실행하려 합니다: npm install' 같은 안내가 뜨면 내용을 확인하고 Approve(승인) 버튼을 누릅니다. 웹의 ChatGPT와 달리 Codex는 내 컴퓨터에서 실행되어 파일을 직접 만들고 바꾸므로, 이 승인은 중요한 안전장치입니다.

명령이 낯설면 '이 명령이 하는 일을 설명해줘' 라고 물어보고, 의도하지 않은 삭제나 외부 접속이면 거부합니다.

(2) 결과 확인

Codex가 작업을 마치면 프로젝트 폴더에 파일이 생성됩니다. 웹 앱이면 아래처럼 프로젝트 폴더에서 개발 서버를 실행합니다.

터미널(명령 프롬프트)

```
D:\vibe_coding\ch04\wordpad-clone> npm run dev

  VITE v5.4.11  ready in 420 ms

  ➜  Local:   http://localhost:3000/
```

`npm run dev`는 `package.json`의 `scripts.dev`에 적힌 개발 서버(Vite)를 실행합니다. 터미널에 `http://localhost:3000/`같은 주소가 표시되면 브라우저에서 그 주소를 열어 결과물을 확인합니다. Vite의 기본 포트는 5173이지만 이 교재의 프로젝트는 3000으로 지정했습니다.

3장 VS Code에서 Claude Code 사용하기

이 장에서는 Anthropic의 Claude Code 확장 프로그램을 VS Code에 설치하고 구성합니다. Claude Code 확장은 인라인 diff, @-멘션, 계획 검토, 키보드 단축키를 통해 AI 코딩 지원을 제공하며, VS Code에서 Claude Code를 쓰는 권장 방법입니다.

학습목표

- Claude Code 확장의 사전 조건을 확인하고 설치할 수 있다.
- 패널을 여는 여러 방법과 로그인 절차를 익힌다.
- 선택한 코드와@-멘션으로 프롬프트를 보내고 변경 사항을 검토할 수 있다.
- CLAUDE.md로 프로젝트 규칙을 Claude에게 알려 줄 수 있다.

이장의구성

3.1	사전 조건과 설치	요구 사항, 확장 설치, 설치 후 문제 해결
3.2	시작하기	패널 열기, 로그인, 프롬프트 전송, 변경 사항 검토
3.3	CLAUDE.md로 프로젝트 안내하기	프로젝트 규칙 파일 작성과 활용

3.1 사전 조건과 설치

이 절에서는 설치 전에 확인할 사항과 설치 방법을 알아봅니다.

(1) 사전 조건

항목	요구 사항	설명
VS Code	1.94.0 이상	`Help > About`에서 버전을 확인합니다.
계정	유료Claude 구독 또는Claude Console	Pro, Max, Team, Enterprise 구독 또는Console 계정이면 되며API 키는 필요 없습니다.
타사 공급자	선택	Amazon Bedrock, Google Cloud를 통해 쓰는 경우 별도 설정이 필요합니다.

확장 프로그램에는 채팅 패널용 CLI(명령줄 인터페이스)가 내장되어 있습니다. 다만VS Code의 통합 터미널에서 `claude`명령을 직접 쓰려면 독립 실행형CLI를 따로 설치해야 합니다.

(2) 확장 프로그램 설치

① 링크 또는 확장 보기에서 설치

공식 문서의 'VS Code용 설치' 링크를 누르면 바로 설치됩니다. 또는 VS Code에서 `Ctrl+Shift+X`(macOS는 `Cmd+Shift+X`)로 확장 보기를 열고'Claude Code'를 검색한 뒤 설치를 클릭합니다. Cursor 등 VS Code 포크에도 설치할 수 있으며, 설치할 수 없는 편집기에서는 CLI를 설치하여 통합 터미널에서 `claude`를 실행합니다.

② 버전 번호의 의미

확장 프로그램의 버전 번호는 함께 번들된 Claude Code의 버전입니다. 특정 기능이'Claude Code v2.1.x 이상'을 요구하면 같은 번호 이상의 확장 버전이 필요하며, 확장 보기에서 확인할 수 있습니다.

(3) 설치 후 확장이 보이지 않을 때

VS Code를 다시 시작하거나, 명령 팔레트(`Ctrl+Shift+P`)에서'Developer: Reload Window'를 실행합니다.

3.2 시작하기

이 절에서는 Claude Code 패널을 열고 로그인한 뒤, 첫 프롬프트를 보내고 변경 사항을 검토하는 전체 흐름을 익힙니다.

(1) Claude Code 패널 열기

VS Code에서 Spark 아이콘은 Claude Code를 나타냅니다. 아래 방법 중 편한 것을 사용합니다.

방법	위치/단축키	비고
편집기 도구 모음	편집기 오른쪽 위의 Spark 아이콘	파일을 열었을 때만 나타납니다.
활동 표시줄	왼쪽 사이드바의 Spark 아이콘	세션 목록을 열며 항상 표시됩니다.
명령 팔레트	`Ctrl+Shift+P` 후 'Claude Code' 입력	'새 탭에서 열기' 등을 고를 수 있습니다.
상태 표시줄	오른쪽 아래✻ Claude Code	파일을 열지 않았을 때도 동작합니다.

Claude 패널은 드래그하여 VS Code의 원하는 위치로 옮길 수 있습니다.

(2) 로그인

처음 패널을 열면 로그인 화면이 나타납니다. 로그인을 클릭하고 브라우저에서 인증을 완료합니다. 로그인 후에는'Claude Code 배우기' 체크리스트가 나타나며, 보여주기로 진행하거나 X로 닫을 수 있습니다. 다시 열려면 설정의 확장 프로그램〉 Claude Code에서'온보딩 숨기기'를 해제합니다.

셸에 `ANTHROPIC_API_KEY`환경 변수가 있는데도 로그인 화면이 계속 나오면 VS Code가 셸 환경을 물려받지 못한 것입니다. 터미널에서 `code .`로 VS Code를 시작하거나 Claude 계정으로 로그인합니다.

(3) 프롬프트 보내기

코드나 파일에 대해 설명, 디버깅, 수정을 요청합니다. Claude는 현재 선택한 텍스트를 자동으로 인식합니다. 선택한 상태에서 `Alt+K`(macOS는 `Option+K`)를 누르면 프롬프트에 `@file.ts#5-10` 같은 @-멘션이 삽입되어 특정 파일의 줄 범위를 정확히 가리킬 수 있습니다.

ch03/prompts/first_prompt.txt

```
React와Vite로 워드패드 클론을 만들어 주세요.
- 굵게, 기울임, 밑줄, 글자 크기 조절 기능
- 파일 저장(.html)과 열기(.html, .txt) 기능
```

```
- React와Vite만 사용하고 다른UI 라이브러리는 추가하지 않습니다.
- 프로젝트 폴더 이름은wordpad-clone 입니다.
- 코드를 작성하기 전에 먼저 계획을 보여 주세요.
```

이 프롬프트는 첫 줄에서 대상 기술을, 목록에서 기능과 제약을 나열하고, 마지막 줄에서 코드 작성 전에 계획을 먼저 보여 달라고 요청합니다. 계획 검토 단계를 거치면 의도와 다른 방향으로 작업이 진행되는 것을 일찍 막을 수 있습니다.

(4) 변경 사항 검토

변경 사항의 표시 방식은 프롬프트 입력창 아래에 표시된 권한 모드에 따라 다릅니다.

권한 모드	동작	권장 상황
수동	편집 전에 원본과 제안 내용을 나란히 보여 주고 수락, 거부, 지시 변경 중 선택하게 합니다.	처음 사용할 때, 중요한 코드를 고칠 때
자동 편집	작업 공간의 대부분 파일을 묻지 않고 편집합니다.	결과를 나중에 한꺼번에 검토할 수 있을 때
계획(Plan)	코드를 바꾸기 전에 계획을 먼저 보여 주고 승인을 받습니다.	큰 작업을 시작할 때

수동 모드의 diff 화면에서는 수락하기 전에 제안된 내용을 직접 고칠 수 있고, Claude는 사용자가 수정했다는 사실을 알게 됩니다. 변경을 하나씩 검토하려면 각 변경 아래의‘이 변경 사항 수락’과‘이 변경 사항 거부’ 버튼을 사용합니다. 변경이 100개를 넘는 diff는 변경별 버튼 없이 열리므로 파일 단위로 검토합니다.

3.3 CLAUDE.md로 프로젝트 안내하기

이 절에서는 프로젝트 루트에 `CLAUDE.md` 파일을 두어, 대화를 새로 시작할 때마다 같은 설명을 반복하지 않도록 하는 방법을 알아봅니다. Claude Code는 이 파일을 읽고 프로젝트의 규칙을 따릅니다.

(1) CLAUDE.md 작성

Claude Code 패널에서 `/init`을 입력하면 프로젝트를 분석하여 초안을 만들어 주며, 직접 작성해도 됩니다. 아래는 이 교재 프로젝트의 파일입니다. 기술 스택, 자주 쓰는 명령, 코딩 규칙을 짧게 적는 것이 핵심입니다.

ch04/wordpad-clone/CLAUDE.md

```
# Wordpad Clone 프로젝트 안내

## 기술 스택
- React 18 + Vite 5 (JavaScript, JSX)
- 외부UI 라이브러리는 사용하지 않는다.

## 자주 쓰는 명령
- 개발 서버 실행: npm run dev (http://localhost:3000)
- 배포용 빌드: npm run build

## 코딩 규칙
- 컴포넌트는 함수형 컴포넌트와Hook만 사용한다.
- 파일 저장/열기 로직은src/fileio.js 에 둔다.
- 새 패키지를 설치하기 전에 먼저 사용자에게 물어본다.
- 주석과 설명은 한국어로 작성한다.
```

'기술 스택'은 Claude가 엉뚱한 라이브러리를 고르지 않게 하고, '자주 쓰는 명령'은 실행과 빌드 방법을 알려 줍니다. '코딩 규칙'의 마지막 두 줄은 패키지 설치 전 확인과 한국어 주석을 요구합니다. 파일이 길어질수록 지시가 흐려지므로 꼭 필요한 규칙만 적습니다.

4장 실습: 워드패드 클론과 두 도구 비교

이 장에서는 2장과 3장의 프롬프트로 만들 수 있는 워드패드 클론의 완성본을 직접 살펴보고 실행합니다. 소스 코드를 이해하면 AI가 만든 코드를 검토하고 수정을 지시할 수 있으며, 마지막으로 Codex와 Claude Code를 비교하고 흔한 문제의 해결법을 정리합니다.

학습목표

- React + Vite 프로젝트의 파일 구조를 설명할 수 있다.
- 각 소스 파일의 역할과 핵심 코드를 이해한다.
- npm install, npm run dev, npm run build를 실행하고 결과를 확인할 수 있다.
- 상황에 맞게Codex와Claude Code를 선택하고 문제를 해결할 수 있다.

이장의 구성

4.1	프로젝트 구조	폴더 구성과 파일별 역할
4.2	소스 코드	설정 파일, 진입점, 파일 입출력, 화면, 스타일
4.3	실행과 확인	패키지 설치, 개발 서버, 빌드, 동작 확인
4.4	비교와 문제 해결	두 도구 비교표, 자주 겪는 문제

4.1 프로젝트 구조

이 절에서는 프로젝트의 폴더 구성을 살펴봅니다. 프로젝트 위치는 `ch04/wordpad-clone`이며, 이후 명령은 이 폴더에서 실행합니다.

(1) 폴더와 파일

경로	역할	작성 위치
package.json	프로젝트 이름, 실행 명령, 필요한 패키지 목록	프로젝트 루트
vite.config.js	Vite 개발 서버 설정(포트3000)	프로젝트 루트
index.html	앱이 붙는HTML 문서와 진입 스크립트 연결	프로젝트 루트
CLAUDE.md	Claude Code용 프로젝트 안내(3.3절)	프로젝트 루트
src/main.jsx	React 앱을 화면에 연결하는 진입점	src 폴더
src/fileio.js	파일 저장/열기 함수 모음	src 폴더
src/App.jsx	도구 모음과 편집기 화면, 동작 로직	src 폴더
src/App.css	화면 모양(스타일)	src 폴더

(2) 데이터 흐름

사용자가 버튼을 누르면 `App.jsx`의 함수가 편집기(`contentEditable`요소)의 내용을 바꾸거나, `fileio.js`의 함수를 호출하여 파일을 저장·열기합니다. 파일은 서버를 거치지 않고 브라우저 안에서만 처리됩니다.

4.2 소스 코드

이 절에서는 프로젝트의 모든 소스 파일을 하나씩 살펴봅니다. 파일은 모두 `vibe_coding.zip`에 그대로 들어 있습니다.

(1) 설정 파일

① package.json

ch04/wordpad-clone/package.json

```
{
  "name": "wordpad-clone",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.4",
    "vite": "^5.4.11"
  }
}
```

`"type": "module"`은 `.js`파일을ES 모듈로 해석하게 합니다. `scripts`의 `dev`, `build`, `preview`는 각각 개발 서버 실행, 배포용 빌드, 빌드 결과 미리보기를 뜻합니다. `dependencies`는 실행에 필요한 React이고, `devDependencies`는 개발에만 필요한 Vite와 React 플러그인입니다. 버전 앞의 `^`는 같은 주 버전 안에서 새 버전을 허용한다는 뜻입니다.

② vite.config.js

ch04/wordpad-clone/vite.config.js

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
```

```
  server: {
    port: 3000,
    open: true,
  },
});
```

`plugins: [react()]`로 JSX를 변환하는 플러그인을 켭니다. `server.port`는 개발 서버 포트를 3000으로 지정하며(기본값은5173), `open: true`는 서버가 시작되면 브라우저를 자동으로 엽니다.

③ index.html

ch04/wordpad-clone/index.html

```
<!doctype html>
<html lang="ko">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Wordpad Clone</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

`<div id="root">`는 React가 화면을 그리는 자리입니다. `<script type="module" src="/src/main.jsx">`가 앱의 진입점을 불러옵니다. `lang="ko"`와 `charset="UTF-8"`은 한글 문서를 위한 설정입니다.

(2) 진입점과 파일 입출력

① src/main.jsx

ch04/wordpad-clone/src/main.jsx

```
import React from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

`createRoot`로 `root`요소에 React 앱을 연결하고 `App` 컴포넌트를 그립니다. `React.StrictMode`는 개발 중 잠재적 문제를 찾아 주는 도우미이며 화면에는 영향을 주지 않습니다.

② src/fileio.js

ch04/wordpad-clone/src/fileio.js

```
// 파일 저장/열기와 관련된 함수를 모아 둔 모듈

export function escapeHtml(text) {
  return text
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;");
}

// 일반 텍스트(.txt)를 편집기에 넣을HTML로 변환한다.
export function textToHtml(text) {
  return text
    .split(/\r?\n/)
    .map((line) => "<div>" + (escapeHtml(line) || "<br>") + "</div>")
    .join("");
}

// 열어 온HTML에서 스크립트와 이벤트 속성을 제거한다.
export function sanitizeHtml(html) {
  const doc = new DOMParser().parseFromString(html, "text/html");
  doc.querySelectorAll("script, iframe, object, embed").forEach((el) => el.remove());
  doc.body.querySelectorAll("*").forEach((el) => {
    Array.from(el.attributes).forEach((attr) => {
      if (attr.name.toLowerCase().startsWith("on")) {
        el.removeAttribute(attr.name);
      }
    });
  });
  return doc.body.innerHTML;
}

function readFileAsText(file) {
  return new Promise((resolve, reject) => {
    const reader = new FileReader();
    reader.onload = () => resolve(String(reader.result));
    reader.onerror = () => reject(reader.error);
    reader.readAsText(file, "utf-8");
  });
}

// 선택한 파일을 읽어 편집기에 넣을HTML 문자열을 돌려준다.
export async function openDocument(file) {
  const text = await readFileAsText(file);
  if (/\.html?$/i.test(file.name)) {
```

```
    return sanitizeHtml(text);
  }
  return textToHtml(text);
}

// 편집기 내용을.html 파일로 내려받는다.
export function saveHtmlFile(fileName, bodyHtml) {
  const name = /\.html?$/i.test(fileName) ? fileName : fileName + ".html";
  const content =
    '<!DOCTYPE html>\n<html lang="ko">\n<head>\n<meta charset="UTF-8">\n' +
    "<title>" + escapeHtml(name) + "</title>\n</head>\n<body>\n" +
    bodyHtml + "\n</body>\n</html>\n";
  const blob = new Blob([content], { type: "text/html;charset=utf-8" });
  const url = URL.createObjectURL(blob);
  const link = document.createElement("a");
  link.href = url;
  link.download = name;
  document.body.appendChild(link);
  link.click();
  link.remove();
  URL.revokeObjectURL(url);
}
```

escapeHtml은 &, <, >를 안전한 문자로 바꿉니다. textToHtml은.txt 내용을 줄마다 로 감쌉니다. sanitizeHtml은 열어 온 HTML에서 script등 위험한 요소와 onclick같은 이벤트 속성을 제거하여, 다른 사람이 만든 파일을 열어도 코드가 실행되지 않게 합니다. openDocument는 확장자가 .html이면 정리한HTML을, 그 밖에는 텍스트를 변환하여 돌려줍니다. saveHtmlFile은 내용을 완전한 HTML 문서로 감싸 Blob으로 만들고, 보이지 않는 링크를 눌러 파일로 내려받게 합니다.

(3) 화면과 스타일

① src/App.jsx

ch04/wordpad-clone/src/App.jsx

```
import { useRef, useState } from "react";
import { openDocument, saveHtmlFile } from "./fileio.js";
import "./App.css";

const FONT_SIZES = [10, 12, 14, 16, 18, 24, 32];

export default function App() {
  const editorRef = useRef(null);
  const fileInputRef = useRef(null);
  const [fontSize, setFontSize] = useState(14);
  const [fileName, setFileName] = useState("untitled.html");
```

```
  const [charCount, setCharCount] = useState(0);

  const updateCount = () => {
    setCharCount(editorRef.current.innerText.replace(/\n/g, "").length);
  };

  const applyFormat = (command) => {
    editorRef.current.focus();
    document.execCommand(command, false, null);
  };

  const handleNew = () => {
    if (!window.confirm("현재 내용을 지우고 새 문서를 만들까요?")) return;
    editorRef.current.innerHTML = "";
    setFileName("untitled.html");
    updateCount();
  };

  const handleOpen = async (event) => {
    const file = event.target.files[0];
    if (!file) return;
    try {
      editorRef.current.innerHTML = await openDocument(file);
      setFileName(file.name.replace(/\.[^.]+$/, "") + ".html");
      updateCount();
    } catch (error) {
      window.alert("파일을 열 수 없습니다: " + error.message);
    }
    event.target.value = "";
  };

  const handleSave = () => {
    saveHtmlFile(fileName, editorRef.current.innerHTML);
  };

  const handleKeyDown = (event) => {
    if ((event.ctrlKey || event.metaKey) && event.key.toLowerCase() === "s") {
      event.preventDefault();
      handleSave();
    }
  };

  // 버튼을 눌러도 편집기의 선택 영역이 해제되지 않게 한다.
  const keepSelection = (event) => event.preventDefault();

  return (
    <div className="app">
      <header className="toolbar">
        <button onMouseDown={keepSelection} onClick={handleNew}>새 문서</button>
```

```
        <button onMouseDown={keepSelection} onClick={() => fileInputRef.current.click()}>
          열기
        </button>
        <button onMouseDown={keepSelection} onClick={handleSave}>저장</button>
        <span className="divider" />
        <button className="b" onMouseDown={keepSelection} onClick={() => applyFormat("bold")}>
          B
        </button>
        <button className="i" onMouseDown={keepSelection} onClick={() => applyFormat("italic")}>
          I
        </button>
        <button className="u" onMouseDown={keepSelection} onClick={() =>
applyFormat("underline")}>
          U
        </button>
        <span className="divider" />
        <label>
          글자 크기
          <select value={fontSize} onChange={(e) => setFontSize(Number(e.target.value))}>
            {FONT_SIZES.map((size) => (
              <option key={size} value={size}>{size}px</option>
            ))}
          </select>
        </label>
        <input
          className="file-name"
          value={fileName}
          onChange={(e) => setFileName(e.target.value)}
          aria-label="파일 이름"
        />
        <input
          ref={fileInputRef}
          type="file"
          accept=".html,.htm,.txt"
          hidden
          onChange={handleOpen}
        />
      </header>
      <main className="page">
        <div
          ref={editorRef}
          className="editor"
          contentEditable
          suppressContentEditableWarning
          role="textbox"
          aria-multiline="true"
          style={{ fontSize: fontSize + "px" }}
          onInput={updateCount}
          onKeyDown={handleKeyDown}
```

```
        />
      </main>
      <footer className="status">
        {fileName} · 글자 수{charCount} · 글자 크기{fontSize}px
      </footer>
    </div>
  );
}
```

useRef로 편집기와 숨겨진 파일 선택 입력을 직접 가리키고, useState로 글자 크기, 파일 이름, 글자 수를 관리합니다. applyFormat은 브라우저의 document.execCommand로 선택 영역에 굵게·기울임·밑줄을 적용합니다. 이 명령은 공식적으로는 더 이상 권장되지 않지만 모든 주요 브라우저에서 동작하므로 학습용으로 사용합니다. keepSelection은 버튼을 눌러도 편집기의 선택 영역이 풀리지 않게 합니다. handleOpen은 선택한 파일을 읽어 편집기에 채우고, handleSave와 Ctrl+S(handleKeyDown) 는 .html파일로 저장합니다. 글자 크기는 편집기 전체에 적용됩니다.

② src/App.css

ch04/wordpad-clone/src/App.css

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: "Malgun Gothic", "Apple SD Gothic Neo", sans-serif;
  background: #d9dde3;
}

.app {
  display: flex;
  flex-direction: column;
  height: 100vh;
}

.toolbar {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 8px 12px;
  background: #f3f4f6;
  border-bottom: 1px solid #b8bdc6;
}

.toolbar button {
  min-width: 36px;
  padding: 4px 10px;
```

```
  cursor: pointer;
}

.toolbar .b { font-weight: bold; }
.toolbar .i { font-style: italic; }
.toolbar .u { text-decoration: underline; }

.divider {
  width: 1px;
  height: 22px;
  background: #b8bdc6;
}

.file-name {
  margin-left: auto;
  width: 200px;
  padding: 4px 8px;
}

.page {
  flex: 1;
  overflow: auto;
  padding: 24px;
}

.editor {
  max-width: 820px;
  min-height: 100%;
  margin: 0 auto;
  padding: 40px 48px;
  background: #ffffff;
  box-shadow: 0 1px 4px rgba(0, 0, 0, 0.25);
  outline: none;
  line-height: 1.6;
}

.status {
  padding: 4px 12px;
  font-size: 12px;
  background: #f3f4f6;
  border-top: 1px solid #b8bdc6;
}
```

`.app`은 화면 전체를 세로 방향 flex로 나누어 도구 모음, 편집 영역, 상태 표시줄을 배치합니다. `.page`는 회색 배경에서 스크롤이 되는 영역이고 `.editor`는 흰 종이처럼 보이는 편집기입니다. `.b`, `.i`, `.u`는 각각 굵게·기울임·밑줄 버튼의 글자 모양을 바꿉니다.

4.3 실행과 확인

이 절에서는 패키지를 설치하고 개발 서버를 실행하여 앱의 동작을 확인한 뒤, 배포용으로 빌드합니다. 모든 명령은 `ch04/wordpad-clone`폴더에서 실행합니다.

(1) 패키지 설치

프로젝트 폴더로 이동하여 `package.json`에 적힌 패키지를 내려받습니다. 인터넷 연결이 필요하며, 처음 한 번만 하면 됩니다.

터미널(명령 프롬프트)

```
D:\vibe_coding> cd ch04\wordpad-clone
D:\vibe_coding\ch04\wordpad-clone> npm install

added 68 packages, and audited 69 packages in 12s

found 0 vulnerabilities
```

`node_modules`폴더가 생기고 패키지가 설치되며, 개수와 시간은 환경에 따라 다릅니다.

(2) 개발 서버 실행과 동작 확인

터미널(명령 프롬프트)

```
D:\vibe_coding\ch04\wordpad-clone> npm run dev

  VITE v5.4.11  ready in 420 ms

  ➜  Local:   http://localhost:3000/
  ➜  press h + enter to show help
```

`open: true` 설정 덕분에 브라우저가 자동으로 열립니다. 열리지 않으면 표시된 주소를 직접 입력합니다. 아래 순서로 동작을 확인합니다.

- 편집 영역에 문장을 입력하고 일부를 드래그로 선택한 뒤 B, I, U 버튼을 눌러 서식이 적용되는지 확인합니다.
- '글자 크기' 목록에서24px 등을 고르면 편집기 전체 글자가 커지는지 확인합니다.
- '저장'을 누르거나 `Ctrl+S`를 누르면 `untitled.html`이 내려받아지는지 확인합니다.
- '새 문서'로 내용을 지운 뒤'열기'에서 방금 저장한 파일을 열어 내용과 서식이 복원되는지 확인합니다.

(3) 배포용 빌드

터미널(명령 프롬프트)

```
D:\vibe_coding\ch04\wordpad-clone> npm run build

vite v5.4.11 building for production...
✓ 32 modules transformed.
dist/index.html                   0.45 kB
✓ built in 1.20s
```

`dist`폴더에 정적 파일이 만들어집니다. 서버는 `Ctrl+C`로 멈춥니다.

4.4 비교와 문제 해결

이 절에서는 두 도구의 특징을 비교하고 실습 중 자주 겪는 문제의 해결법을 정리합니다.

(1) Codex와Claude Code 비교

항목	Codex	Claude Code
제공 회사	OpenAI	Anthropic
로그인 계정	ChatGPT 계정	Anthropic 계정(유료 구독 또는Console)
변경 검토	명령 승인 중심(Approve)	인라인diff, 계획 검토, 줄 범위@-멘션
프로젝트 안내	프롬프트에 직접 기술	CLAUDE.md로 규칙 저장
권장 프롬프트 언어	영어 권장	한국어 사용 가능

결과는 사람이 검토해야 하며, 기능과 비용, 익숙함에 따라 고르거나 같은 프롬프트로 둘을 비교해 봅니다. 화면과 지원 사양은 자주 바뀌므로 막히면 공식 문서를 확인합니다.

(2) 자주 겪는 문제

증상	원인	해결 방법
확장 아이콘이 보이지 않음	설치 직후 창이 갱신되지 않음	VS Code 재시작 또는 `Developer: Reload Window`
npm을 인식하지 못함	Node.js 미설치 또는 터미널 미재시작	Node.js 설치 후 터미널·VS Code를 다시 열기
3000번 포트가 이미 사용 중	다른 프로그램이 같은 포트 사용	터미널에 표시된 새 주소 사용 또는 포트 변경
로그인 화면이 반복됨	환경 변수 상속 문제 등	터미널에서 `code .`로 실행하거나 계정으로 로그인
Windows에서Codex 동작이 불안정	실험적 지원 단계	WSL 환경에서 폴더를 열어 사용